Organizational Meeting Assistant Project Overview & Vision

Document 1 of 3 · Capstone / Graduation Project · AI-Native Product Design · Repository: github.com/Rvin-zh/meeting-assistant · Changelog

The **Organizational Meeting Assistant** turns the raw conversation of a meeting — typed transcript or uploaded/recorded audio — into **structured, actionable knowledge**: a clean summary, key points, explicit decisions, follow-up tasks, a question-answering layer over the meeting, and one-click issue creation in Jira. It is the first product built on top of a reusable, extensible core we call the *LLM-based Organizational AI Core*.

Contents

1. The Problem

Organizations of every size run a constant stream of meetings — formal ones (board meetings, project steering, client reviews) and informal ones (daily stand-ups, team syncs, ad-hoc coordination). These meetings consume a large share of an organization's most

expensive resource, the attention of its people, yet they very often produce **no durable, trackable output**.

The recurring failure modes are well known:

- **Decisions evaporate.** A decision is made out loud, everyone nods, and a week later nobody remembers exactly what was agreed or why.
- **Action items get lost.** Tasks are scribbled in personal notes, scattered across chat threads, or never written down — so ownership and deadlines slip.
- **Meetings are not searchable.** When a question comes up months later ("did we ever decide to drop feature X?"), there is no way to query past discussions.
- **Tooling is fragmented.** Calendar, email, chat, and the project tracker live in separate silos, and the manual copy-paste work of keeping them aligned rarely happens.

The net effect is wasted time, forgotten commitments, unowned responsibilities, and decisions that are never followed through. As organizations push toward agility and automation, the absence of a reliable "meeting memory" becomes a measurable drag on productivity.

2. Product Vision

One-line message	A reusable, extensible organizational AI assistant core that is personalized on each organization's own data.
Technical name	`LLM-based Organizational AI Core`
Market name	A platform for building dedicated, in-house AI assistants for organizations.
First product (this MVP)	The Meeting Assistant — transcript → summary · tasks · RAG · Jira.

The strategic bet is that most organizations do not want a generic chatbot; they want an assistant that understands *their* meetings, *their* projects, and *their* tools. The Meeting Assistant is the wedge: a high-frequency, high-pain workflow that demonstrates the value of the core and produces the proprietary data (decisions, tasks, organizational documents) that makes later capabilities defensible.

3. What the MVP Does Today

The current, demonstrable product delivers a complete end-to-end flow:

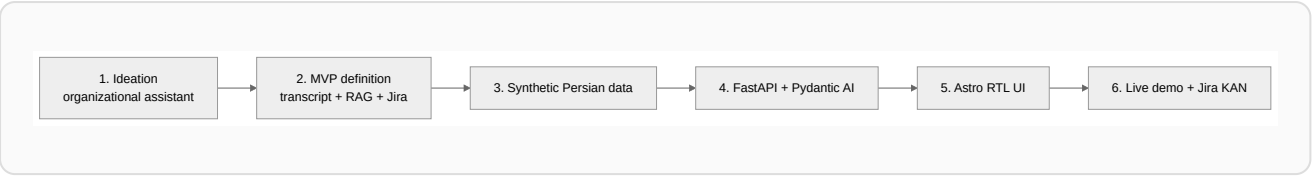
- **Flexible input** — paste a Persian transcript with speaker labels, upload a text file, or provide **audio** (upload `mp3/wav/m4a/ogg/webm/flac` or record directly from the microphone in the browser).
- **Audio transcription** — uploaded or recorded audio is transcribed by Gemini through the same `GOOGLE_API_KEY`, producing an editable transcript before analysis.
- **Automatic analysis** — extraction of a **summary**, **key points**, explicit **decisions**, and structured **tasks** (title, assignee, deadline, priority).
- **Ask-the-meeting (RAG)** — a natural-language Q&A layer scoped to a single meeting, with source attribution.
- **Smart conversational behavior** — small talk such as "hello" is answered directly without searching the transcript; only genuine questions trigger retrieval.
- **Jira integration** — preview and create issues in a Jira Cloud project (issues are written in **English** for clean left-to-right display, while the UI stays Persian).
- **Facilitator guidance** — after a meeting, the assistant offers coaching-style suggestions for running the next meeting better (see Document 2 and 3).

> **Deliberate MVP scope.** This is a demo-ready product, not a finished platform. Live streaming speech-to-text *during* a meeting, professional speaker diarization, and multi-user authentication are intentionally out of scope for the MVP and are planned in the roadmap (Document 3).

4. Academic Framing: An AI-Native Product

Beyond being a working tool, this project is structured as a case study in **AI-native product design** — the journey from idea to a deployable MVP for a product whose core value *is* the AI capability, rather than a conventional app with AI bolted on.

4.1 From idea to MVP



Each stage mirrors a discipline of modern AI product engineering: choosing a problem where an LLM has a genuine edge; defining the smallest feature set that proves the value; preparing representative data; selecting a lightweight, typed agent framework; designing a usable interface; and validating against concrete acceptance criteria.

4.2 Why RAG and an agentic framework

Two ideas from contemporary applied-AI practice anchor the design:

- **Retrieval-Augmented Generation (RAG)** grounds the assistant's answers in the actual meeting content instead of the model's parametric memory, which keeps responses faithful and attributable.
- **Agentic, typed orchestration** (via Pydantic AI) gives every model call a strict output schema, so the system produces validated data structures — not free-form text that downstream code has to guess at.

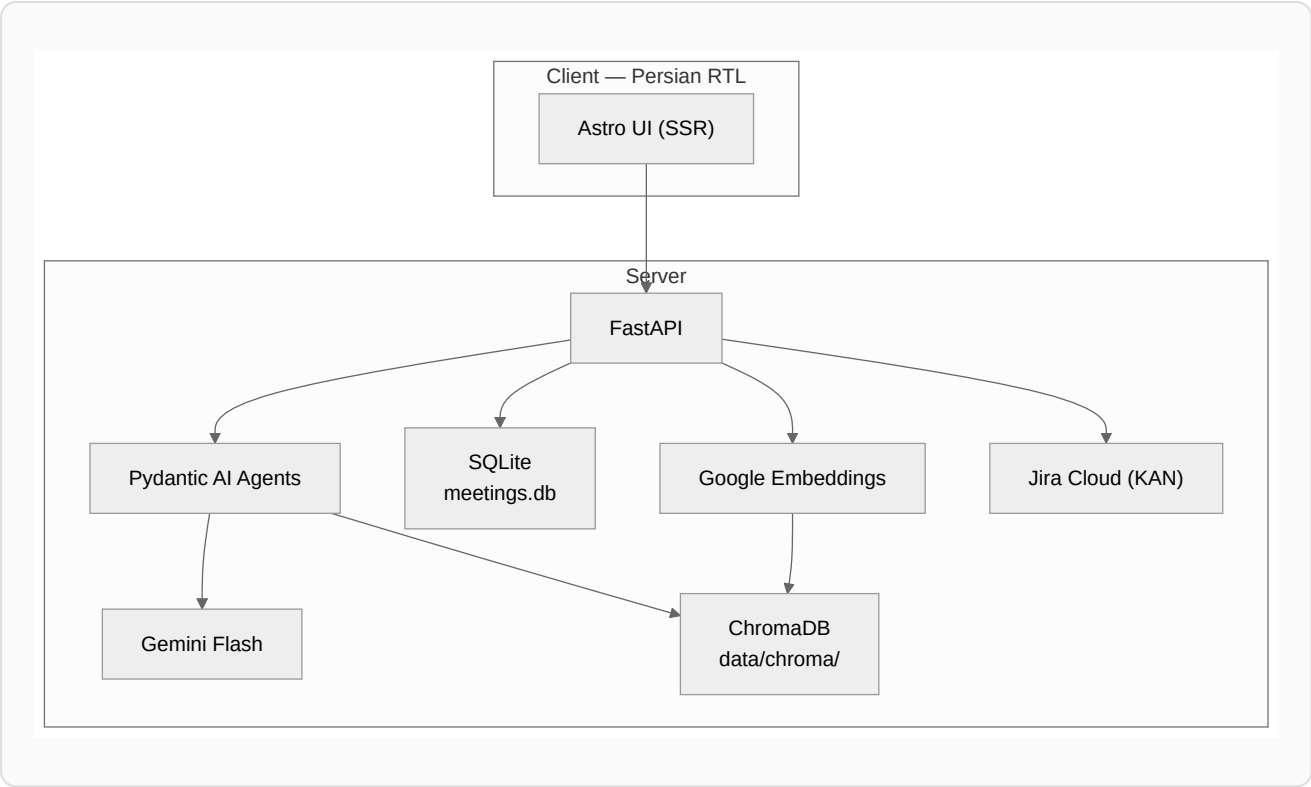
We deliberately chose a *lightweight* agent framework over a heavier orchestration stack: the MVP needs typed outputs and native Gemini integration, not a large dependency surface.

4.3 The product's place in the MLOps / AIOps lifecycle

The MVP focuses on the build-and-deliver phase, but the design anticipates the full operational lifecycle of an AI product — continuous data collection, prompt and model evaluation, deployment, and monitoring. These concerns are scoped into the roadmap (Document 3) rather than implemented in the MVP, which is the honest and realistic stance for a capstone deliverable.

5. System Architecture

The system is a small, cleanly separated set of services: an Astro (SSR) front end speaking to a FastAPI back end, which orchestrates Pydantic-AI agents over Gemini, persists meetings in SQLite, stores vector embeddings in ChromaDB, and integrates with Jira Cloud.



The full request/response data flow, the database schemas, the API surface, and the user interface are documented in detail in Document 2 — Technical Implementation.

6. Agentic Design

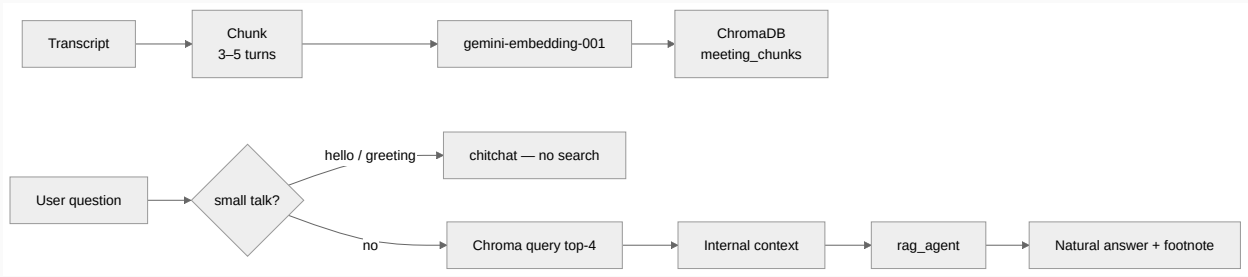
Each AI responsibility is encapsulated in a small, single-purpose agent with a typed output contract. This keeps prompts focused, makes outputs testable, and lets the system grow by adding agents rather than enlarging one monolithic prompt.

Agent	Responsibility	Status
`analysis_agent`	Transcript → structured `MeetingAnalysis` (summary, key points, decisions, tasks).	Implemented
`rag_agent`	Answer a question using retrieved meeting context, with attribution and no raw transcript dump.	Implemented
`facilitation_agent`	Coaching-style suggestions for running better meetings, from analysis + participation stats.	Implemented

`alignment_agent`	Compare decisions against an uploaded SOW / contract (in-scope vs out-of-scope).	Planned
`sentiment_agent`	Per-speaker tone analysis over transcript text, with ethics/RBAC guardrails.	Planned

7. Retrieval-Augmented Generation (RAG)

When a meeting is ingested, its transcript is parsed into turns, grouped into small chunks (3–5 turns), embedded with `gemini-embedding-001`, and indexed in ChromaDB under the meeting's id. A question is answered by retrieving the most relevant chunks and letting the `rag_agent` compose a natural answer grounded in them.



Aspect	MVP (today)	Future
Search scope	One meeting	Multi-meeting archive + project filters
Vector store	**ChromaDB** (embedded, `data/chroma/`)	Chroma Cloud / pgvector / Vertex
Chunking	By conversation turn	Semantic chunking with overlap

8. Technology Stack

Layer	Choice	Why

Front end	Astro (SSR) + lightweight CSS, RTL	Server-rendered meeting pages; no heavy SPA framework needed.
Back end	FastAPI + Python 3.11+	Async, typed, excellent for AI service glue.
Agent framework	Pydantic AI	Typed outputs, native Gemini, minimal dependency surface.
LLM	`google:gemini-3.5-flash`	Fast, multimodal (handles audio transcription too), cost-effective.
Meeting store	SQLite (`data/meetings.db`)	Zero-ops persistence for the MVP; migration path to PostgreSQL.
Vector store	ChromaDB (embedded)	No separate server; fast HNSW similarity search.
Embeddings	`gemini-embedding-001`	Same provider as the LLM; high-quality multilingual vectors.
Issue tracking	Jira Cloud (project `KAN`)	Industry-standard target for action items.

9. Data & Feature Engineering

The MVP ships with three synthetic Persian meetings (a stand-up, a planning session, and a sprint review) so the product can be demonstrated and tested without any real recordings. From each transcript the system derives a small set of lightweight features that power retrieval and attribution:

Feature	Source	Use
`speaker`	Transcript parsing	RAG citation and assignee hints
`timestamp`	Parsed `[HH:MM]`	Chronological ordering of chunks
`turn_count`	Chunk metadata	Context-window sizing
`embedding`	Google API	Similarity search

10. Evaluation & Success Criteria

The MVP is considered successful when the following are demonstrably true:

- Three synthetic Persian meetings are processed end-to-end.
- Summary and tasks are produced at acceptable quality.
- At least three RAG questions are answered naturally with source footnotes.
- Audio transcription (upload or recording) flows into the same analysis pipeline.
- At least one issue is created in Jira.
- The UI is fully Persian and right-to-left.
- The automated test suite is green — **150 passing unit tests** (13 more skip without optional/live credentials) plus a **13-test live API & integration suite**.

A hypothetical A/B evaluation frames the value question for the academic report: version A shows summary and tasks only; version B adds the RAG tab. The hypothesis is that B reduces the time to find a past decision and increases the rate of tasks pushed to Jira — measured by time-to-answer, citation accuracy, and Jira conversion.

11. Where This Is Going

> The product roadmap (Sprints 2–6+) and the advanced, differentiating capabilities — the facilitator guide, SOW/contract alignment, sentiment analysis, multi-meeting organizational memory, and the MLOps quality loop — are described in Document 3 — Roadmap & Future Work. The concrete engineering details of the current system are in Document 2 — Technical Implementation.